

DURA-CPS: A Multi-Role Orchestrator for Dependability Assurance in LLM-Enabled Cyber-Physical Systems

1st Trisanth Srinivasan

Department of AI Deployment and Safety

Cyrion Labs

Dallas, USA

trisanth@cyrionlabs.org

1st Santosh Patapati

Department of AI Deployment and Safety

Cyrion Labs

Dallas, USA

santosh@cyrionlabs.org

2nd Himani Musku

School of Computer Science

Carnegie Mellon University

Pittsburgh, USA

hmusku@andrew.cmu.edu

3rd Idhant Gode

College of Engineering

Cornell University

Ithaca, USA

iag32@cornell.edu

4th Aditya Arora

ACM AI

University of California San Diego

San Diego, USA

a7arora@ucsd.edu

5th Samvit Bhattacharya

Department of R&D

Cyrion Labs

San Roman, USA

samvitb@cyrionlabs.org

6th Abubakr Nazriev

Sentinel DE

University of Montana

Missoula, USA

abu.nazriev@umconnect.unt.edu

7th Sanika Hirave

Dept. of Computer Science & Engineering

Oakland University

Rochester, USA

sanikahirave@oakland.edu

8th Zaryab Kanjiani

School of Applied Economics and Management

Cornell University

Ithaca, USA

zk226@cornell.edu

9th Srinjoy Ghose

School of Science

University of North Texas

Denton, USA

srinjoyghose@my.unt.edu

Abstract—Cyber-Physical Systems (CPS) increasingly depend on advanced AI techniques to operate in critical applications. However, traditional verification and validation methods often struggle to handle the unpredictable and dynamic nature of AI components. In this paper, we introduce DURA-CPS, a novel framework that employs multi-role orchestration to automate the iterative assurance process for AI-powered CPS. By assigning specialized roles (e.g., safety monitoring, security assessment, fault injection, and recovery planning) to dedicated agents within a simulated environment, DURA-CPS continuously evaluates and refines AI behavior against a range of dependability requirements. We demonstrate the framework through a case study involving an autonomous vehicle navigating an intersection with an AI-based planner. Our results show that DURA-CPS effectively detects vulnerabilities, manages performance impacts, and supports adaptive recovery strategies, thereby offering a structured and extensible solution for rigorous V&V in safety- and security-critical systems.

Index Terms—Cyber-Physical Systems (CPS), Verification and Validation (V&V), Artificial Intelligence (AI), Safety-Critical Systems, Large Language Models (LLM), Autonomous Driving

I. INTRODUCTION

Cyber-Physical Systems (CPS) integrate computational algorithms with physical processes. CPS is being used across various sectors such as transportation, energy, manufacturing, healthcare, and agriculture. Ensuring that these systems are safe, secure, reliable, and timely is critical as failures may result in serious consequences. Thus, Verification and Validation (V&V) is essential to build trust that a CPS meets

its requirements and behaves as intended. V&V remains a complex and costly endeavor in most domains [1].

The challenges of V&V grow when Artificial Intelligence (AI) is incorporated into CPS. This difficulty is compounded as increasingly complex AI techniques, such as Deep Neural Networks (DNNs) and Large Language Models (LLMs) are deployed in these systems. The behavior of AI is often sensitive to unexpected changes in the environment [2]. This unpredictability, coupled with a vulnerability to adversarial attacks [3], makes traditional V&V methods, which rely on predictable behavior and exhaustive state exploration [4], less effective for assessing AI-based components.

Addressing these issues requires V&V frameworks designed specifically for AI-based CPS. Such frameworks should validate AI models in isolation and how they interact with the environment and other system components at runtime. We require structured approaches that can systematically:

- Evaluate AI behavior in context against diverse dependability criteria
- Assess robustness to noise, faults, and security threats
- Facilitate the analysis of interactions between AI, the physical system, and its environment
- Support iterative refinement
- Provide traceable evidence suitable for building assurance cases and supporting certification [5]

To meet these needs, we present DURA-CPS, a novel frame-

work centered around multi-role orchestration to iteratively assure the dependability of AI components within CPS. DURA-CPS assigns roles to various agents (AI models, algorithms, formal checkers, etc.) to work together within a simulated CPS environment. The agents continuously assess, challenge, and refine the performance of the primary AI component according to the defined dependability requirements.

The contributions of the proposed framework are as follows:

- 1) **Multi-Role V&V Architecture:** Defines specialized roles (e.g., 'Generator', 'SafetyMonitor', 'SecurityAssessor', 'PerformanceOracle', 'FaultInjector', 'RecoveryPlanner') tailored for comprehensive dependability assessment.
- 2) **Iterative Assurance Loop:** Implements a controlled feedback loop where V&V findings from monitoring/assessment roles inform subsequent actions, test generation, or adaptation planning.
- 3) **Simulation Integration & State Management:** Provides interfaces for connecting to standard CPS simulators and manages the shared state information needed for contextual V&V.
- 4) **Extensibility:** Designed for modularity, allowing users to define new roles or customize existing ones with different AI models, formal techniques, or procedural logic.
- 5) **Dependability-Focused Metrics:** Collects metrics specifically related to safety violations, security vulnerabilities detected, performance adherence, and recovery effectiveness.

By orchestrating these roles in a coordinated manner, DURA-CPS aims to provide a more rigorous approach to V&V for AI in CPS than ad-hoc testing or isolated component analysis.

II. RELATED WORK

The assurance of AI-based CPS draws upon research within several fields.

A. Verification and Validation of CPS

Traditional V&V approaches for CPS are based on formal methods such as model checking and theorem proving [6], simulation-based testing [7], hardware-in-the-loop validation, runtime verification [8], [9], and fault or attack injection [10]. These methods form the backbone of CPS assurance. However, they face challenges when applied to complex AI components because of issues with scalability, lack of test coverage, and difficulties in modeling the unpredictable behavior of AI [2].

B. Verification and Validation of AI/ML Systems

Research on AI V&V focuses on robustness testing against perturbations and adversarial examples [11], [12], as well as the formal verification of network properties using techniques such as SMT solvers or abstract interpretation [13], [14]. Efforts have also been made to improve explainability [15], [16] and assess fairness in machine learning systems [17]. Simulation-based test generation is also commonly used to

supplement offline analyses [18], [19]. However, these approaches tend to target specific individual properties. They do not capture the full complexity of runtime interactions in a dynamic CPS environment, such as chain reaction events [20].

C. Runtime Assurance and Monitoring

Runtime Assurance (RA) techniques aim to ensure safety during operation by employment monitors and safety controllers [21], [22]. For example, the Simplex architecture [21] switches between a complex primary controller and a simpler, verified safety controller as needed. Runtime verification methods assess execution traces against formal specifications using temporal logic such as LTL, MTL, or STL [23], [24]. In contrast to these approaches, DURA-CPS utilizes multiple coordinated roles in a closed-loop assurance process. This goes beyond just switching to backup controllers.

D. Multi-Agent Systems (MAS) for Simulation and V&V

Multi-agent systems have been used to simulate complex systems and test control strategies in CPS [25]. Agent-based modeling has been used to mimic CPS behavior which focuses on autonomous driving scenarios. These approaches simply utilize agent interactions for simulation. On the other hand, DURA-CPS assigns specific V&V roles that target dependability assurance rather than general-purpose simulation or emergent behavior analysis.

E. LLM and AI Orchestration Frameworks

Recent frameworks such as LangChain [26] and LlamaIndex [27] have shown how to compose LLM calls with external tools and data to build applications. AutoGen [28] further supports complex workflows involving multiple LLM agents. Although these orchestration frameworks demonstrate considerable power, they are primarily designed for application development and general AI collaboration. Their capabilities for formal requirement checking, CPS simulation, and structured assurance loops are limited compared to the dedicated design of DURA-CPS. This often forces researchers to develop custom assurance loops and V&V steps from the ground up, which is highly time consuming. Earlier work, such as the LLMOrchestrator [29], provided a basic generator-verifier structure that DURA-CPS significantly expands by incorporating multiple V&V roles.

F. Positioning DURA-CPS

DURA-CPS builds on ideas from the aforementioned fields while establishing a new niche. It adopts the orchestration paradigm from AI frameworks, re-purposing it specifically for dependability V&V. The framework employs a multi-role architecture inspired by multi-agent systems. It assigns roles that focus on safety monitoring, security testing, performance evaluation, fault injection, and recovery planning. Unlike approaches that rely solely on runtime monitoring and simple controller switching, DURA-CPS implements an iterative, closed-loop process that integrates tightly with CPS simulation environments as a robust solution for the complex challenges of V&V in AI-based CPS.

III. THE DURA-CPS FRAMEWORK

DURA-CPS is a Python-based framework designed to structure and automate the iterative V&V process for AI components within simulated CPS environments. It employs a multi-role orchestration paradigm where specialized computational agents, termed Roles, collaborate to assess the behavior of the primary AI component Under Test (AUT) against dependability requirements.

A. Core Architecture

The DURA-CPS architecture, illustrated in Fig. 1, centers around an Orchestration Controller that manages the interaction between various Roles, an Environment Interface connecting to the CPS simulator, a State Manager maintaining shared state, and a Dependability Metrics tracker.

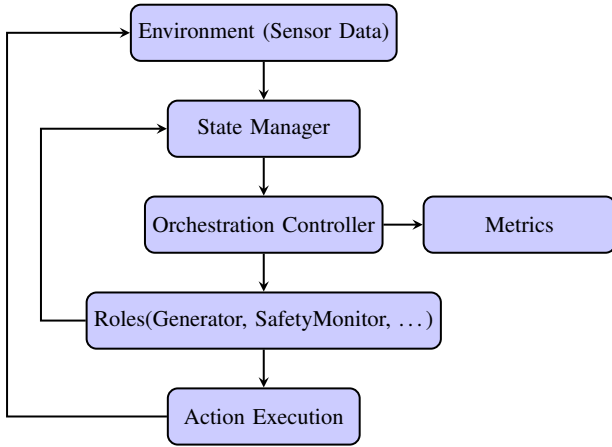


Fig. 1. Overview of the DURA-CPS architecture. Sensor data from the CPS is collected by the Environment Interface and organized by the State Manager. The Orchestration Controller coordinates specialized Roles that generate and refine actions through a dedicated Action Execution module. These actions are fed back into the system to complete a closed-loop assurance process, while Metrics are concurrently tracked for continuous verification and validation

B. Key Components

DURA-CPS consists of five key components.

1) *Orchestration Controller*: This central component manages the overall execution flow. It initializes the roles, manages the iterative V&V loop, sequences role execution based on dependencies or triggers, facilitates communication between roles (via the State Manager), and terminates the process based on predefined criteria (e.g., number of iterations, test completion, violation detected).

2) *Role*: A Role represents a specialized function within the V&V process. It's an abstract base class defining a standard interface. Users implement or configure concrete Role subclasses. Roles interact indirectly through a State Manager. Key predefined (but extensible) roles include:

- **Generator**: Represents the primary AI component Under Test (AUT) or a component generating inputs/scenarios for it. Takes current state/context, generates an action, plan, or output.

- **SafetyMonitor**: Checks the state, proposed actions, or predicted outcomes against safety rules or invariants. Can use rule-based logic, formal specifications (e.g., STL checks via integrated monitors like RTAMT [30]), or even another AI model trained for safety assessment. Returns a safety verdict (e.g., safe, unsafe, warning) and potentially quantitative scores.
- **SecurityAssessor**: Evaluates the system's security posture. Can analyze potential vulnerabilities based on the current state or AI output, or direct the FaultInjector. Might involve checking against known attack patterns or security policies.
- **PerformanceOracle**: Monitors performance metrics against requirements (e.g., response time, resource usage, control accuracy, task completion metrics).
- **FaultInjector**: Introduces faults or disturbances into the simulation based on directives (e.g., from the SecurityAssessor or predefined test plans). Can simulate sensor noise/failure, communication delays/loss, GPS spoofing, or adversarial perturbations to AI inputs.
- **RecoveryPlanner**: Activated upon detection of safety/security violations or critical failures. Proposes recovery actions or adaptations (e.g., switch to safe mode, replan trajectory, trigger alert). Can be rule-based or use planning algorithms/AI.

3) *Environment Interface*: Provides an abstraction layer for communicating with the external CPS simulator (e.g., CARLA, AirSim, Gazebo [31]–[33]). It handles:

- Sending commands/actions (from Generator or RecoveryPlanner) to the simulator.
- Receiving sensor data and state updates from the simulator.
- Translating between the simulator's data formats and the internal state representation.
- Potentially controlling simulation time steps or scenario loading.

4) *State Manager*: Maintains the shared state accessible by all roles. This includes:

- Current state received from the Environment Interface (e.g., vehicle position, sensor readings).
- Outputs produced by roles in the current iteration (e.g., the Generator's proposed action, the SafetyMonitor's verdict, the FaultInjector's active fault).
- Historical state information if needed for temporal analysis.

Ensures consistent view of the system state for all roles within an iteration.

5) *DependabilityMetrics*: Collects and logs key metrics throughout the orchestration process, such as:

- Number and type of safety/security violations detected.
- Performance metric values over time.
- Robustness scores from monitors (if applicable).
- Fault injection success/impact.
- Recovery action success rates.
- Processing time per role/iteration.

This data is crucial for post-hoc analysis and generating assurance reports.

C. Iterative Orchestration Workflow

The DURA-CPS workflow can be customized as needed. A typical execution cycle proceeds as follows:

- 1) **Initialization:** Controller loads configuration, initializes roles, connects to the simulator via the Environment Interface, and gets initial state via the State Manager.
- 2) **Iteration Start:** Controller triggers roles based on sequence or dependencies.
- 3) **State Update:** The Environment Interface provides current world state to the State Manager.
- 4) **Generation or Action Proposal:** The Generator (AUT) proposes an action based on current state.
- 5) **Dependability Assessment:**
 - SafetyMonitor evaluates proposed action/state against safety rules.
 - SecurityAssessor evaluates security posture; may direct the FaultInjector.
 - PerformanceOracle checks performance metrics.
 - FaultInjector potentially introduces faults/attacks based on directives or test plan.
- 6) **V&V Feedback Processing:** Controller gathers verdicts/outputs from assessment roles via the StateManager.
- 7) **Decision and Adaptation:**
 - If violations detected: Controller may halt, log details, or activate the RecoveryPlanner. The RecoveryPlanner proposes alternative action.
 - If no violations: Controller approves Generator action (or refined action).
- 8) **Action Execution:** Controller sends the final approved or recovery action to the simulator via the EnvironmentInterface.
- 9) **Metrics Logging:** Relevant data is logged through DependabilityMetrics for the iteration.
- 10) **Loop/Terminate:** Controller checks termination conditions (e.g., time limit, scenario end, critical failure). If not met, proceeds to the next iteration (Step 2).

This loop allows for continuous evaluation and adaptation, forming an iterative assurance process within the simulation. The configuration of roles, their interaction logic (dependencies and triggers), and the connection to the simulation environment define the specific V&V experiment conducted using the framework.

D. Extensibility

DURA-CPS is designed for extensibility. Users can:

- Implement new Role subclasses using custom Python code, integrating different AI models (LLMs, DNNs), formal verification tools (via wrappers), or standard algorithms.
- Define complex interaction protocols and triggering conditions between roles.



Fig. 2. A sample CARLA 3D scene. A third-person view was used as input for our Llama 3.2 11B model alongside sensor readings. [31], [34], [35]

- Develop new EnvironmentInterface subclasses to support different simulators or hardware-in-the-loop setups.
- Customize the DependabilityMetrics collection.

This allows tailoring the framework to specific CPS domains, AI components, and V&V requirements.

IV. USE CASE: AUTONOMOUS VEHICLES

To demonstrate the capabilities of DURA-CPS, we apply it to a challenging V&V scenario: ensuring the safe and secure navigation of an unsignalized urban intersection by an autonomous vehicle (AV) equipped with an AI-based planning module.

A. Scenario Description

The AV must navigate a four-way intersection potentially shared with other vehicles (simulated background traffic) and pedestrians. The primary AI component Under Test (AUT) is an LLM-based tactical planner. Given sensor inputs (object lists, positions, velocities from simulated perception) and a high-level goal (e.g., "proceed straight"), the LLM planner generates maneuver decisions (e.g., "wait", "accelerate", "yield", "proceed cautiously"). Critical dependability requirements include:

- **Safety:** Avoid collisions with other vehicles and pedestrians. Maintain safe following distances. Obey traffic rules implicitly (e.g., right-of-way, though not explicitly programmed).
- **Security:** Resilience against spoofed sensor data (e.g., ghost obstacles, false trajectories) intended to cause hazardous behavior or gridlock.
- **Performance:** Navigate the intersection without undue delay (avoiding excessive conservatism) or comfort violations (jerk/acceleration).

1) *Rationale for an LLM-Based Planner:* Existing AV planning stacks use domain-specific rule sets. While this is effective, it limits our ability to stress-test DURA-CPS across diverse AI use cases. We therefore deliberately use an LLM-based planner, which is relatively weaker in this area, to 1) show the framework can wrap any black-box decision module, 2) surface failure modes that traditional frameworks would not

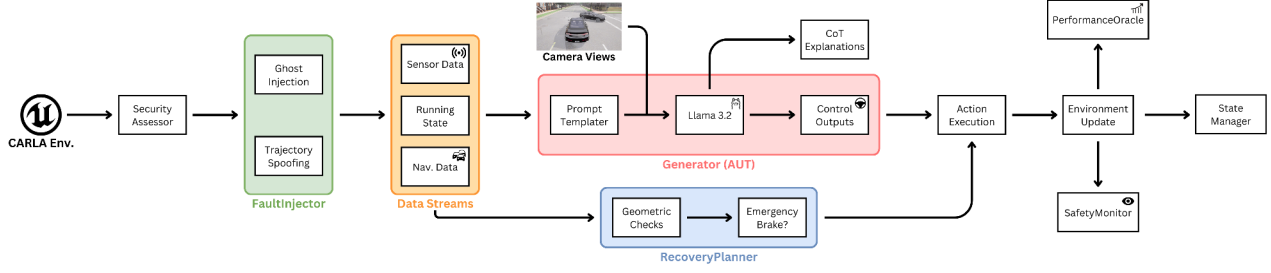


Fig. 3. High-level orchestration of AI-based generator and other roles within DURA-CPS. The CARLA environment supplies navigation (map, traffic, waypoints) and sensor data. The SecurityAssessor can inject faults through the FaultInjector. These data streams, alongside the running state, feed into a prompt templater to generate a textual representation for the Llama 3.2 11B model. Camera views are passed directly to the LLM. Llama 3.2 generates both control outputs and corresponding explanations. The Action Execution module then applies these outputs to the environment. In parallel, the RecoveryPlanner employs geometric checks and determines whether or not to employ the emergency brake, which overrides all other actions. The running state is updated via the StateManager to include past actions and associated CoT explanations. The PerformanceOracle and SafetyMonitor track for performance.

find, and 3) to demonstrate how DURA-CPS's different roles interact when the AI's internal logic is opaque.

B. DURA-CPS Configuration

We configured DURA-CPS with the following roles instantiated for this scenario (Figure 3):

- **Generator (AUT):** An LLM (fine-tuned Llama 3.2 11B variant [34]) prompted with current perceived world state and goal, outputting a tactical maneuver decision. The LLM is provided few-shot examples and a Chain-of-Thought (CoT) prompt. Table I details the sensor inputs the LLM receives.
- **SafetyMonitor:** Implemented using geometric checks and simplified traffic rules. It verifies if the proposed maneuver maintains a minimum safety distance from all perceived dynamic objects based on predicted trajectories. Flags "unsafe" if violations are predicted.
- **SecurityAssessor:** Monitors incoming sensor data patterns. For this use case, it directs the FaultInjector to periodically introduce specific attacks.
- **FaultInjector:** Simulates two attack types based on SecurityAssessor triggers:
 - 1) *Ghost Obstacle Injection:* Adds a non-existent dynamic obstacle into the perceived state provided to the Generator.
 - 2) *Trajectory Spoofing:* Modifies the predicted velocity or path of a real detected vehicle to appear more hazardous than it is.
- **PerformanceOracle:** Tracks intersection clearance time and maximum longitudinal/lateral acceleration/jerk. Flags "performance_fail" if thresholds are exceeded.
- **RecoveryPlanner:** A simple rule-based agent. Using the same geometric checks as the SafetyMonitor, it flags checks for unsafe conditions. If unsafe conditions are detected, it overrides the Generator's decision with "emergency_brake".

1) *Integration:* The CARLA simulator [31] was used via a custom DURA-CPS CarlaInterface (Figure 2). The StateManager tracked perceived object lists (from CARLA's sensors), proposed actions, and associated CoT explanations.

2) *Orchestration Logic:* The controller executed roles sequentially within each simulated time step (100ms), where processing is aligned to 100 ms of simulated time, in the following order: Environment Update, Generator, SafetyMonitor, SecurityAssessor, FaultInjector (conditional), PerformanceOracle, Decision (Controller activates RecoveryPlanner if unsafe), Action Execution (Figure 1).

C. Test Scenarios

We designed simulation scenarios with varying complexity and injected faults/attacks:

- 1) **Nominal:** Light traffic, clear right-of-way.
- 2) **Congested:** Moderate traffic density, requiring careful yielding and gap selection.
- 3) **Conflicting Traffic:** Vehicles approaching simultaneously from multiple directions, testing navigation logic.
- 4) **Ghost Obstacle Attack:** Nominal scenario + FaultInjector adds a ghost obstacle near the intersection entry.
- 5) **Trajectory Spoofing Attack:** Congested scenario + FaultInjector spoofs the trajectory of an oncoming car to seem aggressive.
- 6) **Pedestrian Crossing:** Scenario with a simulated pedestrian crossing the AV's intended path.

Each scenario was run 15 times with variations in traffic patterns and timing.

D. Expected Outcomes and Metrics

We used DURA-CPS to assess:

- **Safety Violations:** Frequency of the SafetyMonitor flagging "unsafe" maneuvers (indicating potential collisions based on geometry/rules). Actual collisions logged by CARLA serve as ground truth confirmation.
- **Security Resilience:** How the Generator (LLM) reacts to injected faults. Does it behave erratically, freeze (grid-lock), or follow unsafe commands induced by fake data? Frequency of SafetyMonitor activations during attacks.
- **Performance Degradation:** Increase in intersection clearance time or comfort violations (jerk/acceleration) under congestion or attacks.

TABLE I
CARLA SENSOR INPUTS FOR USE CASE ARCHITECTURE

Sensor Input	Description
LiDAR-based Obstacle Summary	Textual summary of obstacles extracted from the LiDAR. Instead of using raw 3D data, the CarlaIn-terface aggregates nearby objects (vehicles, pedestrians, static obstacles) with positions & dimensions.
Radar Summary	A text summary of radar detections that includes each object's range and relative radial velocity.
Front RGB Camera	An RGB image captured from the front-facing camera passed directly to the LLM.
Third-Person View Camera	An RGB image providing a broader, third-person perspective of the intersection. This image delivers contextual clues about background traffic and environmental layout.
IMU Summary	A text-based summary of inertial measurements that includes linear acceleration, angular velocity, and heading. This information succinctly describes the vehicle's motion dynamics.
Vehicle Speed	A numerical value representing the current speed of the vehicle, extracted from vehicle odometry.
HD Map & Waypoint Data	A structured list of upcoming way points or lane center coordinates derived from a high-definition map. This input supports high-level route planning and navigation.
Traffic Controls Status	A concise textual report detailing the state of nearby traffic signals and the presence of key road signs.

- **Recovery Effectiveness:** Success rate of the Recovery-Planner (emergency brake) in preventing actual collisions when activated by the SafetyMonitor.

Metrics were collected by the DependabilityMetrics compo-nent.

V. RESULTS AND ANALYSIS

This section presents the results from executing the au-tonomous intersection navigation use case with DURA-CPS across the defined scenarios (15 runs per scenario, total 90 runs).

A. Safety Assessment

The SafetyMonitor actively checked the LLM Generator's proposed maneuvers. Table II summarizes the percentage of runs where the SafetyMonitor flagged at least one "un-safe" prediction and the rate of actual collisions observed in CARLA.

TABLE II
SAFETY MONITOR ACTIVATIONS AND COLLISION RATES

Scenario Type	Monitor Flags "Unsafe" (%)	Collision Rate (%)
Nominal	6.7% (1/15)	0.0% (0/15)
Congested	20.0% (3/15)	6.7% (1/15)
Conflicting Traffic	33.3% (5/15)	13.3% (2/15)
Ghost Obstacle Attack	86.7% (13/15)	6.7% (1/15)
Trajectory Spoof Attack	60.0% (9/15)	20.0% (3/15)
Pedestrian Crossing	26.7% (4/15)	6.7% (1/15)
Overall Avg.	38.9%	8.9%

Observations:

- The LLM planner showed reasonable safety in nominal cases but struggled increasingly with complexity (Con-gested, Conflicting Traffic). It exposed weaknesses that validated the need for our multi-role loop.
- Security attacks significantly triggered the SafetyMonitor. The Ghost Obstacle attack frequently caused the LLM to propose sudden braking or swerving deemed unsafe by the monitor. Trajectory Spoofing often led the LLM to yield unnecessarily or hesitate, sometimes causing conflicts later flagged by the monitor.

- Actual collisions were lower than monitor flags, primarily because the RecoveryPlanner (emergency brake) often intervened successfully when triggered by the Safety-Monitor. The cases where collisions still occurred despite monitor flags often involved very short time-to-collision where braking was insufficient or complex multi-vehicle interactions.

DURA-CPS effectively identified scenarios where the LLM planner proposed potentially unsafe actions.

B. Security Assessment and Resilience

The FaultInjector, directed by the SecurityAssessor, suc-cessfully introduced faults. Analysis focused on the LLM's reaction:

- **Ghost Obstacle:** The LLM consistently reacted to the ghost obstacle despite the visual input contradicting sen-sor input. It would often propose immediate braking or significant deceleration, treating it as real. This frequently led to performance issues (sudden stops, increased clear-ance time) and was often flagged by the SafetyMonitor if the braking was excessively abrupt.
- **Trajectory Spoofing:** The LLM was sensitive to the spoofed aggressive trajectories, typically choosing to yield or wait much longer than necessary, significantly impacting performance. In 3 runs (20%), this excessive caution led to situations where the AV became 'stuck', unable to find a perceived safe gap, resulting in a gridlock scenario broken only by simulation timeout.

DURA-CPS's roles allowed systematic injection and observa-tion of the AI's vulnerability to sensor data manipulation.

C. Performance Impact

The PerformanceOracle tracked intersection clearance time and comfort metrics. Figure 4 shows average clearance time.

As expected, congestion and conflicting traffic increased clearance time. Security attacks had a major impact: Ghost Obstacles caused sharp braking, sometimes increasing time due to recovery, while Trajectory Spoofing significantly in-creased waiting times due to the LLM's overly cautious reaction to the fake aggressive behavior. Comfort violations

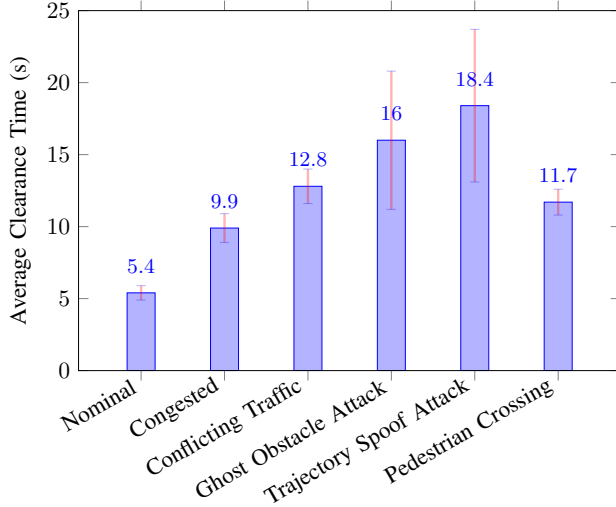


Fig. 4. Average Intersection Clearance Time Across Scenarios. Error bars indicate the standard deviation over 15 simulation runs.

(high jerk/acceleration) were most frequent during recovery braking and reactions to ghost obstacles.

D. Recovery Effectiveness

The simple RecoveryPlanner (emergency brake) was triggered whenever the SafetyMonitor flagged "unsafe".

- It successfully prevented a collision in cases where it was activated and a collision would likely have occurred otherwise (based on manual inspection of near-miss scenarios).
- Failures typically occurred when the unsafe situation developed too rapidly for braking alone to suffice or involved complex side-impact scenarios.

This highlights the importance of the monitor-recovery loop but also suggests the need for more sophisticated recovery strategies than simple braking in future work.

E. DURA-CPS Contribution Analysis

This use case shows how DURA-CPS facilitates V&V:

- The multi-role setup allowed assessment of safety, security, and performance aspects of the LLM planner.
- The FaultInjector and SecurityAssessor enabled systematic testing against specific attack vectors.
- The closed-loop orchestration revealed interactions, such as security attacks leading to safety monitor triggers, or recovery actions impacting performance.
- DependabilityMetrics provided quantitative data summarizing complex behaviors across runs and scenarios.

The framework provided a systematic way to challenge the AI component and evaluate its dependability.

VI. DISCUSSION

The results from the autonomous intersection navigation use case provide insights into the challenges of assuring AI de-

pendability in complex CPS and demonstrate the utility of the DURA-CPS framework's multi-role orchestration approach.

A. Implications for AI/LLM Assurance in CPS

The experiment shows the brittleness of even sophisticated AI models like LLMs when faced with complex scenarios and threats. The LLM planner showed degraded safety and performance under congestion, conflicting goals, and especially under simulated attacks. Its sensitivity to spoofed data (ghost obstacles and manipulated trajectories) is a significant concern, showing that relying solely on the AI's perceived world model without robust validation mechanisms is high-risk.

Security attacks directly impacted safety (by inducing unsafe reactions or hesitation leading to secondary conflicts) and performance (by causing excessive caution or gridlock). This demonstrates the need for V&V frameworks like DURA-CPS that can assess these attributes concurrently and analyze their interactions, rather than treating them in isolation. The partial success of the simple recovery mechanism shows the importance of runtime assurance loops but also shows the need for more advanced monitoring and recovery strategies tailored to specific failure modes.

B. Effectiveness and Role of DURA-CPS

DURA-CPS proved effective in structuring this complex V&V task. Its key strengths observed in the use case include:

- **Structured Assessment:** Assigning specific dependability concerns (safety, security, performance) to distinct roles provided clarity and allowed for modular implementation of checks and attacks.
- **Systematic Fault/Attack Injection:** The FaultInjector role enabled controlled introduction of security threats, allowing systematic evaluation of the AI's resilience.
- **Closed-Loop Analysis:** The framework captured the dynamic feedback loop where AI actions influence the environment, which influences subsequent AI inputs and V&V assessments, including recovery actions.
- **Extensibility Potential:** While simple monitors and recovery were used here, the role-based structure readily accommodates more sophisticated implementations (e.g., STL-based monitors, AI-based security assessors).

DURA-CPS acts as an "in-the-loop V&V orchestrator," allowing engineers to configure a virtual team of specialized agents that continuously probe and assess the AI AUT within its simulated operational context. This approach facilitates identifying weaknesses, understanding interactions between dependability facets, and evaluating the effectiveness of assurance mechanisms like monitors and recovery planners.

C. Limitations and Future Work

This work has several limitations that suggest avenues for future research:

- **Specification Effort:** Defining logic for each role (especially monitors and assessors) requires great effort and domain expertise. Developing libraries of reusable role implementations would improve usability.

- **Sim-to-Real Gap:** As with all simulation-based V&V, transferring findings to the real world requires caution. Validating DURA-CPS and role implementations on physical platforms or high-fidelity digital twins is crucial.
- **Scalability:** Orchestrating many complex roles, especially those involving computationally intensive analysis or multiple AI inferences per time step, could become a bottleneck. In the simulated environment, we executed the AI agents at each simulated interval rather than in real-time. Such an approach would not be possible in direct real-world testing. In such cases, performance optimization and distributed execution may be needed.
- **LLM Specifics:** While an LLM was used as the AUT, deeper analysis of LLM-specific failure modes (e.g., hallucination, prompt sensitivity) within the CPS context could be integrated into specific roles.

Future directions focus on:

- 1) Developing a richer library of predefined V&V roles incorporating diverse techniques (STL monitoring, simplified formal methods, ML-based anomaly detection). One approach may be to automatically generate V&V roles using LLMs given the problem and constraints.
- 2) Integrating DURA-CPS with hardware-in-the-loop (HIL) setups to evaluate its effectiveness.
- 3) Applying DURA-CPS to other domains, including industrial robotics and dependable agricultural automation (e.g., safe human-robot collaboration in smart farming).
- 4) Investigating optimizations and removing bottlenecks that limit DURA-CPS's effectiveness in out-of-sim tests.
- 5) Developing specialized assessment metrics tailored to LLM-specific failure modes, such as hallucination, and integrating these into the DURA-CPS framework to improve LLM-based component assurance.

VII. CONCLUSION

This paper introduces DURA-CPS, a framework leveraging multi-role orchestration to structure and automate the iterative assurance process for AI-based CPS. By assigning specialized V&V functions to roles within a simulated CPS environment, DURA-CPS systematically evaluates AI behavior against a range of dependability requirements. Its iterative and closed-loop approach allows analysis of complex interactions and continuous evaluation of runtime assurance mechanisms.

We demonstrated the utility of DURA-CPS in a case study on autonomous vehicle intersection navigation with an LLM-based planner. The case study shows how DURA-CPS can effectively inject faults, detect safety and security vulnerabilities, assess performance impacts, and evaluate recovery actions. DURA-CPS provides quantitative metrics for assurance.

In conclusion, DURA-CPS offers a practical method for the V&V of AI in CPS where safety and security are critical. Further work should focus on the identified challenges like specification effort and bridging the simulation-to-reality gap.

REFERENCES

- [1] G. Klein, J. Andronick, K. Elphinstone, T. Murray, T. Sewell *et al.*, "Comprehensive formal verification of an os microkernel," *ACM Trans. Comput. Syst.*, 2014.

- [2] K. Lekadir, R. Osuala, C. Gallina, N. Lazrak *et al.*, "Future-ai: Guiding principles and consensus recommendations for trustworthy artificial intelligence in medical imaging," *arXiv preprint*, 2024.
- [3] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2015.
- [4] E. M. Clarke *et al.*, *Model Checking*. MIT Press, 1999.
- [5] R. Hawkins, T. Kelly, J. Knight, and P. Graydon, "A new approach to creating clear safety arguments," in *Advances in Systems Safety*, 2011.
- [6] R. Alur, "Principles of cyber-physical systems," *MIT Press*, 2015.
- [7] C. Birchler, S. Khatiri, P. Rani, T. Kehrer, and S. Panichella, "A roadmap for simulation-based testing of autonomous cyber-physical systems: Challenges and future direction," *arXiv preprint*, 2024.
- [8] M. Leucker and C. Schallhart, "A brief account of runtime verification," *Journal of Logic and Algebraic Programming*, 2009.
- [9] C. Sanchez, G. Schneider *et al.*, "A survey of challenges for runtime verification from advanced application domains (beyond software)," *Formal Methods in System Design*, 2019.
- [10] M.-C. Hsueh *et al.*, "Fault injection techniques & tools," *Computer*, 1997.
- [11] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *IEEE Symposium on Security and Privacy (SP)*, 2017.
- [12] M.-I. Nicolae, M. Sinn, M. N. Tran, A. Rawat, M. Wistuba *et al.*, "Adversarial robustness toolbox v1.0.0," *arXiv preprint*, 2018.
- [13] G. Katz, C. Barrett, *et al.*, "Reluplex: An efficient smt solver for verifying deep neural networks," in *Computer Aided Verification*, 2017.
- [14] T. Gehr, M. Mirman, D. Drachler-Cohen, Tsankov *et al.*, "AI²: Safety and robustness certification of neural networks with abstract interpretation," in *IEEE Symposium on Security and Privacy (SP)*, 2018.
- [15] M. T. Ribeiro, S. Singh, and C. Guestrin, "why should i trust you?": Explaining the predictions of any classifier," in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [16] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Neural Information Processing Systems* 30, 2017.
- [17] N. Mehrabi, F. Morstatter, N. Saxena, Lerman *et al.*, "A survey on bias and fairness in machine learning," *ACM Computing Surveys*, 2021.
- [18] V. Landersheim, M. Jurisch, R. Bartolozzi, G. Stoll, R. Möller, and H. Dann, "Simulation-based testing of subsystems for autonomous vehicles at the example of an active suspension control system," 2022.
- [19] D. Humeniuk, F. Khomh, and G. Antoniol, "Ambiegen: A search-based framework for autonomous systems testing," in *arXiv preprint*, 2023.
- [20] R. Uuk, C. I. Gutierrez, D. Guppy, L. Lauwaert *et al.*, "A taxonomy of systemic risks from general-purpose AI," 2024, arXiv preprint.
- [21] S. Sheikhi, U. Mehmood, S. Bak, S. A. Smolka, and S. D. Stoller, "The black-box simplex architecture for runtime assurance of multi-agent cps," *Innovations in Systems and Software Engineering*, 2024.
- [22] M. Abate, E. Feron, and S. Coogan, "Monitor-based runtime assurance for temporal logic specifications," *arXiv preprint*, 2019.
- [23] O. Maler *et al.*, "Monitoring temporal properties of continuous signals," in *Modeling and Analysis of Timed and Fault-Tolerant Systems*, 2004.
- [24] A. Donzé, "Breach, a toolbox for verification and parameter synthesis of hybrid systems," in *Computer Aided Verification*, 2010.
- [25] M. Ahmed, O. Kazar, A. Ratnayake *et al.*, "Cyber-physical system model based on multi-agent system," *Cyber-Phys. Systems*, 2024.
- [26] H. Chase, "Langchain," 2022, github repository.
- [27] J. Liu, "Llamaindex (gpt index)," 2022, github repository.
- [28] Q. Wu, G. Bansal *et al.*, "Autogen: Enabling next-gen llm applications via multi-agent conversation framework," *arXiv preprint*, 2023.
- [29] T. Srinivasan and S. Patapati, "LLMorchestrator: A multi-model llm orchestration framework for reducing bias and iterative reasoning," 2025.
- [30] D. Nickovic, A. Pant, and G. Schneider, "Rtam: Online robustness monitors from signal temporal logic," in *Runtime Verification*, 2016.
- [31] A. Dosovitskiy *et al.*, "Carla: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, 2017.
- [32] S. Shah and Dey, "Airsim: High-fidelity visual and physical simulation for autonomous vehicles," in *Field and Service Robotics*, 2017.
- [33] N. Koenig *et al.*, "Design paradigms for gazebo, an open-source multi-robot simulator," in *RSJ Conf. on Intelligent Robots and Systems*, 2004.
- [34] Grattafiori *et al.*, "The llama 3 herd of models," 2024, arXiv preprint.
- [35] X. Li, A. Kumar, Sharma *et al.*, "Dialogue-based generation of self-driving simulation scenarios using large language models," 2023, arXiv.